[이커머스 2 조] 연동 명세 (API 명세)

Oct 23, 2026 · @청취사 프로젝트

1. 개요와 공통 규칙

화면이 부르는 백엔드(Spring Boot) API 24 개와, 백엔드가 부르는 FastAPI(계산 엔진 · AI 인사이트), FastAPI 가 부르는 LLM 의 요청·응답 형식을 정한다. 범위는 Must 기능만이다. 기준 문서는 10/19 팀 결정을 반영한 기획서, 요구사항 정의서 최종본, 서비스 플로우, 아키텍처 및 데이터 스키마(최종 ERD: AX_project_v6, 표 16 개), 에이전트 구성도다.

항목	규칙
기본 경로	`/api/v1` (Spring Boot)
인증	보호 API 는 Authorization: Bearer <accessToken>으로 인증한다. 액세스 JWT 는 기본 15 분, 리프레시 토큰은 최초 로그인부터 최대 7 일이다. 리프레시는 HttpOnly 쿠키로 보관하며 로그인·갱신·로그아웃 등 쿠키 인증 POST 에는 CSRF 헤더가 필요하다. 인증 endpoint 와 상세 예외는 3.1·6 장을 따른다.
권한	다른 계정의 상품·파일·계산 결과를 요청하면 `403 FORBIDDEN` (화면·API 직접 호출 모두)
형식	JSON(UTF-8). 수불부 업로드만 `multipart/form-data`
필드 이름	API 는 camelCase, DB 는 snake_case (`skuBarcode` ↔ `sku_barcode`)
날짜	날짜 `YYYY-MM-DD`, 일시 ISO 8601 (`2026-10-19T09:30:00+09:00`)
숫자	수량은 정수(개), 금액은 원 단위 정수, 위험·비율은 0~1 소수(화면에서 %로 표시)
목록 응답	`{"items": [...], "total": 31}`
오류 응답	`{"code": "UPLOAD_DUPLICATE", "message": "...", "details": {...}}` + HTTP 상태 (6 장)
오래 걸리는 작업	정제·계산·평가는 `202 Accepted` + 작업 ID 를 바로 돌려주고, 상태 조회 API 로 진행·완료·실패를 확인한다 (BE-17 처리 상태 관리)

용어: **기준일**(`asOfDate`) = 업로드한 수불부의 마지막 날짜 · **계산 ID**(`runId`) = 한 번의 계산 실행 · **결과 ID**(`resultId`) = 그 실행 안의 SKU 하나의 결과 · **기존 발주 물량**(`plannedReceipt`) = 발주했지만 아직 도착하지 않은 물량(입고 예정).

2. API 목록

Must 24 개다. 화면은 화면 흐름도의 S01~S07 과 팝업 M1~M4, 요구사항은 최종본 ID 기준이다. 계산은 업로드 완료 · 입고 조건 저장 · 기존 발주 저장 때 서버가 자동으로 실행하므로, 화면에서 계산을 시작하는 API 는 없다.

ID	메서드	경로	하는 일	화면	요구사항
A1	POST	`/auth/signup`	회원가입 (회사 정보 · 인증 토큰 포함)	S02 회원가입	FE-01 · BE-01
A2	POST	`/auth/login`	로그인 (JWT·리프레시 쿠키 발급)	S01 로그인	FE-01 · BE-01
A3	POST	`/auth/logout`	로그아웃	공통 (머리 메뉴)	FE-01 · BE-01
A4	GET	`/auth/me`	회원 정보 조회 (담당자 이름·회사명)	공통	BE-02
A5	GET	`/auth/email-availability?email=`	이메일 중복 확인	S02 회원가입	FE-01 · BE-01
A6	POST	`/auth/email-verifications`	인증 코드 메일 발송 · 다시 보내기	S02 회원가입	FE-01 · BE-01
A7	POST	`/auth/email-verifications/confirm`	인증 코드 확인 → 인증 토큰	S02 회원가입	FE-01 · BE-01
U1	POST	`/uploads`	수불부 업로드, 정제·검증 시작	S04 파일 확인 · M1 · M2	FE-02 · BE-03 · FE-03 · BE-04
U2	GET	`/uploads/{uploadId}`	처리 상태 · 자료 기간 · 검증 리포트(제외 상품 이름) · 입고 확인된 발주	S05 업로드 결과	FE-02 · BE-03 · BE-07 · BE-17
U3	GET	`/uploads/periods`	이미 올린 기간 목록 · 최근 업로드 ID	S04 파일 확인 · S03 검증 리포트 링크	FE-02 · FE-03 · BE-04
P1	GET	`/products`	상품 목록 · 검색 · 상품별 입고 조건	S06 입고 조건 · M4	BE-05 · FE-06 · FE-14
P2	PATCH	`/products/{productId}`	상품명·원가·사용 여부 수정	S06 입고 조건	BE-05
S1	GET	`/replenishment`	전체 입고 조건 기본값	S06 입고	FE-06 · BE-

ID	메서드	경로	하는 일	화면	요구사항
		`-settings`	조회	조건	06
S2	PUT	`/replenishment-settings`	전체 기본값 저장 · 계산 시작 (확인하고 계산하기 = 항상 계산)	S06 입고 조건 · M3	FE-06 · BE-06 · BE-08
S3	PUT	`/products/{productId}/replenishment-settings`	상품별 입고 조건 저장 → 그 상품 계산	S06 · S07	FE-06 · BE-06 · BE-08
R1	GET	`/planned-receipts`	이미 발주한 물량 목록 · 도착 상태	S06 입고 조건	FE-06 · BE-07
R2	POST	`/planned-receipts`	이미 발주한 물량 등록 → 계산	M4 기존 발주 등록	FE-06 · FE-11 · BE-07 · BE-08
R3	PATCH · DELETE	`/planned-receipts/{receiptId}`	수정(도착일 고치기 등) · 취소	S06 입고 조건	FE-06 · BE-07
C2	GET	`/calculation-runs/{runId}`	계산 상태 · 완료 요약(지금 넣어야 할 상품 수)	M3 계산 완료·실패	FE-06 · FE-13 · BE-17
D1	GET	`/dashboard/replenishment-needs`	입고 필요 목록 · 정렬 3 가지 · 계산 불가 · 검산 제외 목록 · 갱신 전 여부	S03 대시보드	FE-04 · BE-11 · DA-05
K1	GET	`/products/{productId}/detail`	SKU 상세: 입고 필요 상태 · 입고 조건 · 계산 근거(올림 전 필요량) · 추이 · 정확도 등급	S07 SKU 상세	FE-05 · FE-09 · BE-11 · BE-12 · DA-02 · DA-13
K2	GET	`/results/{resultId}/scenarios`	시나리오 비교표 (80·100·120%)	S07 SKU 상세	FE-07 · BE-09 · DA-06 · DA-07
K3	POST	`/results/{resultId}/scenarios`	직접 입력 수량 시나리오 추가	S07 SKU 상세	FE-07 · BE-09
K4	GET	`/results/{resultId}/insight`	AI 인사이트 문장	S07 SKU 상세	FE-08 · BE-10 · BE-15

3. 화면 ↔ 백엔드 API 상세

각 API 의 요청·응답 본문과 대표 오류다. 숫자는 예시다.

3.1 인증 · 회원가입 (A1 · A2 · A4 · A5 · A6 · A7)

API	요청	성공	오류
A5 `GET /auth/email-availability?email=`	쿼리에 이메일	`200 {"available": true}`	`400 INVALID_INPUT` (형식)
A6 `POST /auth/email-verifications`	`{"email": "seller@a.com"}`	`202 {"expiresIn": 600}` 6 자리 코드를 메일로 발송	`409 EMAIL_EXISTS`, `429 TOO_MANY_REQUESTS` (1 분에 1 번)
A7 `POST /auth/email-verifications/confirm`	`{"email": "...", "code": "482915"}`	`200 {"verificationToken": "vt_..."}` (30 분 유효)	`400 VERIFICATION_FAILED` (5 번 틀리면 코드 무효), `400 VERIFICATION_EXPIRED`
A1 `POST /auth/signup`	`{"companyName": "펫스낵랩", "businessRegNo": null, "managerName": "박셀러", "email": "...", "password": "...", "verificationToken": "vt_..."}`	`201 {"userId": 1, "email": "..."}`	`400 INVALID_INPUT` (details.field: password 등), `400 VERIFICATION_REQUIRED`, `409 EMAIL_EXISTS`
GET /api/auth/csrf	로그인 전에도 호출 가능. 쿠키 포함 요청	200: headerName·parameterName·token	—
A2 `POST /auth/login`	`form-urlencoded: email, password. 쿠키·CSRF 헤더 포함. JSON 로그인 미지원.`	`200: accessToken·tokenType·expiresIn + ipgo_refresh 쿠키. 회원 정보는 /auth/me 로 조회.`	`401 INVALID_CREDENTIALS, 403 FORBIDDEN`
POST /api/auth/refresh	리프레시 쿠키·CSRF 헤더. credentials: include	200: 새 JWT JSON·새 리프레시 쿠키	401: 아래 refresh 오류 코드. 403 FORBIDDEN
POST /api/auth/logout	리프레시 쿠키·CSRF 헤더	204: 현재 로그인 폐기·쿠키 삭제. 반복 호출도 204	403 FORBIDDEN

API	요청	성공	오류
A4 `GET /auth/me`	없음	`200 {"userId": 1, "email": "...", "managerName": "박셀러", "companyName": "펫스낵랩"}`	`401 UNAUTHENTICATED.` 만료·위조·폐기 오류는 6장 참조.

인증 공통 계약과 호출 순서

인증 endpoint의 전체 경로는 /api/auth/...이다. 표의 /auth/... 상대 표기도 같은 인증 경로를 뜻한다. 로그인·갱신 성공 JSON은 아래와 같고, 화면의 회원 정보는 GET /api/auth/me로 조회한다.

`200 {"accessToken":"<서명된 JWT>","tokenType":"Bearer","expiresIn":900}`

1. 로그인 전에 GET /api/auth/csrf를 credentials: include로 호출하고 응답의 headerName·token을 인증 POST 헤더에 넣는다. 로그인은 form-urlencoded로 email·password를 보내며 쿠키를 포함한다. 로그인 성공 후 CSRF 쿠키가 초기화되므로 갱신·로그아웃 전에 CSRF를 다시 받는다.

2. 액세스 JWT는 프론트 메모리에 보관하고 보호 API에 Bearer 헤더로 보낸다. 새로고침 시 CSRF 취득 후 refresh로 복구한다. 일반 Bearer API는 CSRF 헤더가 필요 없고, refresh·logout은 만료된 Bearer가 있어도 쿠키로 처리하지만 CSRF가 필요하다.

3. ACCESS_TOKEN_EXPIRED일 때만 refresh를 한 번 호출하고 성공하면 원래 요청을 한 번 재시도한다. 프론트는 동시 갱신을 하나로 조율한다. 갱신 실패 또는 다른 인증 오류는 반복 갱신하지 않고 재로그인한다.

리프레시 쿠키는 ipgo_refresh, HttpOnly, 배포 시 Secure, SameSite=Lax, Path=/api/auth, Domain 미지정이다. DB에는 SHA-256 해시만 저장한다. 갱신마다 토큰을 교체하고 이전 토큰을 무효화하며 최초 로그인부터 최대 7일인 마감을 연장하지 않는다. 액세스 JWT도 로그인 마감 이내로 제한돼 마지막에는 expiresIn이 900보다 작을 수 있다.

이전 refresh 재사용은 해당 로그인 전체를 폐기하며 다른 독립 로그인은 유지한다. 로그아웃은 현재 로그인에서 발급한 기존 JWT도 commit 후 새 요청부터 거부하고 쿠키를 삭제한다. 이미 실행 중인 요청을 취소하는 기능은 아니다.

머리 메뉴에는 `managerName`(회사 담당자 이름)을 보여준다.

백엔드 검사 조건: 회사명 · 담당자 이름 필수(1~50자) · 사업자등록번호는 선택(입력하면 숫자 10자리) · 이메일 형식 · 중복 없음 · 비밀번호 8자 이상 영문+숫자 · 인증 토큰이 있고 30분 안이며 같은 이메일일 것. 화면 검사는 편의용이고, 최종 판단은 모두 백엔드에서 한다.

3.2 수불부 업로드 (U1 · U2 · U3)

U1 `POST /uploads` — `multipart/form-data`, 필드 `files`(XLSX 여러 개), `overwrite`(기본 `false`).

```
// 202 처음 올린 기간 → 정제·검증 시작
{"uploadId": 12, "status": "PROCESSING",
 "files": [{"originalName": "2026-09.xlsx", "periodStart": "2026-09-01",
"periodEnd": "2026-09-30"}]}

// 409 같은 기간을 다시 올림 → 화면이 덮어쓰기 확인 팝업을 띄우고 overwrite=true로 다시
```

보냄
```
{"code": "UPLOAD_DUPLICATE", "message": "이미 올린 기간입니다",
 "details": {"existing": {"uploadId": 9, "originalName": "2026-09.xlsx",
   "periodStart": "2026-09-01", "periodEnd": "2026-09-30", "uploadedAt":
"2026-10-15T10:02:00+09:00"}}}
```

- 같은 기간 판정: 파일명의 연·월(자료 기간)이 이미 올린 기간과 겹치면 같은 기간으로 보고 `409 UPLOAD_DUPLICATE`와 기존 업로드 건(이름·기간·업로드 일시)을 돌려준다. 내용 해시는 완전히 같은 파일을 감지하는 데만 쓴다(BE-04). 화면은 U3로 올리기 전에 먼저 알려준다.
- 재업로드 확인 팝업(M2)은 Must다. 덮어쓰기를 고르면 화면이 `overwrite=true`로 다시 보내 교체 후 재계산하고, 취소하면 기존 자료를 그대로 둔다(FE-03 · BE-04).
- S04 화면이 고른 파일을 먼저 확인해 M1 팝업을 띄우고, 서버에서도 XLSX가 아니거나 파일명에서 연·월을 못 읽으면 `400 UPLOAD_INVALID_FORMAT`, `details`에 파일명과 사유.

U3 `GET /uploads/periods` — 이미 올린 기간 목록. S04는 고른 파일 이름의 연·월을 이 목록과 비교해 올리기 전에 "이미 올린 기간"을 표시하고 M2를 띄운다(서버도 U1에서 다시 확인해 409). S03의 "검증 리포트 보기"는 `latestUploadId`로 U2를 연다.

```
{"items": [
  {"period": "2026-09", "uploadId": 12, "fileName": "2026-09.xlsx",
"periodStart": "2026-09-01", "periodEnd": "2026-09-30", "uploadedAt": "2026-
10-15T10:02:00+09:00", "status": "DONE"},
  {"period": "2026-10", "uploadId": 15, "fileName": "2026-10.xlsx",
"periodStart": "2026-10-01", "periodEnd": "2026-10-14", "uploadedAt": "2026-
10-15T10:05:00+09:00", "status": "DONE"}],
 "latestUploadId": 15}
```

U2 `GET /uploads/{uploadId}`

```
{"uploadId": 15, "status": "DONE",
 "periodStart": "2026-09-01", "periodEnd": "2026-10-14", "asOfDate": "2026-
10-14",
 "files": [{"fileName": "2026-09.xlsx", "period": "2026-09"}, {"fileName":
"2026-10.xlsx", "period": "2026-10"}],
 "validation": {"checkedSkuCount": 159, "targetSkuCount": 158,
   "excludedSkus": [{"productId": 108, "skuBarcode": "8800272421082",
"skuName": "덴탈껌 M 5입", "date": "2026-09-12", "reason": "전날 재고 + 입고 − 출고
± 조정이 당일 재고와 3개 달라요"}]},
 "matchedReceipts": [{"receiptId": 5, "skuName": "덴탈껌 S 10입", "quantity":
60, "receivedOn": "2026-10-20", "status": "RECEIVED"}],
 "runId": 31}
```

`status`는 `PROCESSING` · `DONE` · `FAILED`. S05는 `periodStart`~`periodEnd`를 자료 기간, `asOfDate`를 기준일, `targetSkuCount`를 계산 대상으로 보여준다. 검산이 맞지 않는 SKU는 계산에서 빠지고 이름 · 날짜 · 사유와 함께 `excludedSkus`에 남는다. `runId`는 정제가 끝나면 서버가 자동으로 시작한 계산이다. 이때 등록된 기존 발주와 새 입고를 맞춰, 입고가 확인된 발주를 `matchedReceipts`로 돌려준다(3.3 규칙). 화면은 "발주 N건이 입고로 확인됐어요"를 보여준다.

3.3 입고 조건 · 기존 발주 물량 (S1~S3 · R1~R3)

```
// P1 GET /products?query=케어&page=0&size=20   (S06 상품별 조건 표 · M4 상품 선택)
{"items": [{"productId": 101, "skuBarcode": "8800272420016", "skuName": "
케어트릿 120g", "unitCost": 1800, "isActive": true,
   "settings": {"leadTimeDays": 5, "reviewCycleDays": 29, "orderUnitQty":
12}, "overridden": ["orderUnitQty"]}],
 "total": 31, "page": 0, "size": 20}
```

```
// S1 GET /replenishment-settings
{"confirmed": false, "leadTimeDays": 5, "reviewCycleDays": 29,
"reviewCycleSource": "ESTIMATED", "orderUnitQty": 1}

// S2 PUT /replenishment-settings   (S06 "확인하고 계산하기" · M3 "다시 시도")
{"leadTimeDays": 5, "reviewCycleDays": 29, "orderUnitQty": 1, "recalculate":
true}
// → 200 {"saved": true, "runId": 34}   recalculate가 true면 값이 같아도 항상
계산을 시작

// S3 PUT /products/101/replenishment-settings
{"leadTimeDays": 5, "reviewCycleDays": 29, "orderUnitQty": 12, "unitCost":
1800}
// → 200 {"saved": true, "runId": 35}

// R2 POST /planned-receipts  (M4 팝업)
{"productId": 101, "quantity": 120, "expectedOn": "2026-12-01"}
// → 201 {"receiptId": 7, "reflected": true, "runId": 36}  저장하면 자동 계산
```

- 리드타임·입고 주기는 1 이상 정수, 입고 단위는 1 이상 정수, 원가는 0 이상. 어기면 `400 INVALID_INPUT`, `details.field`에 칸 이름.
- S1의 confirmed는 셀러가 모든 상품 기본값을 확인·저장했는지다. 처음엔 false라 화면이 "처음 한 번만 확인" 단계를 띄우고, 확인하기 전에는 제안값(리드타임 5일, 수불부로 추정한 입고 주기, 입고 단위 1개)을 돌려준다. 저장한 기본값은 창고(warehouses)에 두며, 이때부터 reviewCycleSource는 ESTIMATED가 아니라 DEFAULT다. 상품별 값이 비어 있으면 기본값을 쓰고, 셀러가 바꾼 칸은 P1의 overridden에 넣는다.
- 도착 예정일 없이 등록하면 저장은 하고 `"reflected": false`를 돌려준다. 화면은 "계산에 반영되지 않음" 안내를 띄운다(FE-11). 예시의 케어트릿 120g 발주(120개, 12/1 도착)는 커버 기간(오늘 10/19부터 34일, 11/22까지) 밖이라 계산에는 반영 대상이지만 이번 권장량에서 빼는 수량(plannedReceiptsInHorizon)은 0이다.

기존 발주 물량 처리 규칙 — 재고는 수불부만 믿고, 발주는 수불부에 입고로 찍히는 순간 계산에서 빠진다. 업로드 간격(주 1회 등)과 상관없이 같은 물량을 두 번 빼지 않는다.

상황	처리	status
도착 예정일이 기준일 이후	계산에 반영 (기준일~오늘 사이 도착분은 예측 구간에서 더함)	PENDING
새 업로드에서 그 상품 입고가 발주 등록 이후에 찍힘	도착 완료로 바꾸고 계산에서 뺌 (입고 확인일 저장)	RECEIVED
예정일이 기준일보다 지났는데 입고가 없음	자동으로 빼지 않고 셀러 확인 요청	OVERDUE
도착 예정일 없음	저장만, 계산 제외	NO_DATE

```
// R1 GET /planned-receipts
{"items": [
  {"receiptId": 5, "productId": 102, "skuName": "덴탈껌 S 10입", "quantity":
60, "expectedOn": "2026-10-24", "receivedOn": "2026-10-20", "status":
"RECEIVED", "reflected": false},
  {"receiptId": 7, "productId": 101, "skuName": "케어트릿 120g", "quantity":
```

```
120, "expectedOn": "2026-12-01", "status": "PENDING", "reflected": true}],
 "total": 2}
```

3.4 계산 상태 (C2)

```
// 계산은 업로드 완료 · S2 · S3 · R2 때 서버가 시작하고, 응답의 runId로 상태를 조회해요.
// "확인하고 계산하기"와 M3 "다시 시도"는 S2를 recalculate: true로 보내요 (값이 같아도
계산).

// C2 GET /calculation-runs/34
{"runId": 34, "status": "DONE", "asOfDate": "2026-10-14", "engineVersion":
"0.3.1",
 "summary": {"needCount": 6, "notNeededCount": 22, "notCalculableCount": 2,
"excludedCount": 1},
 "startedAt": "2026-10-19T09:00:01+09:00", "completedAt": "2026-10-
19T09:00:21+09:00"}

// 실패
{"runId": 34, "status": "FAILED", "errorMessage": "계산 시간이 너무 오래 걸렸어요",
"previousRunId": 31}
```

`status`는 `QUEUED` · `RUNNING` · `DONE` · `FAILED`. 화면은 계산을 직접 시작하지 않고("다시 계산" 버튼 없음) 저장 응답의 `runId`로 C2를 조회한다. DONE이면 M3 완료 팝업에 `summary.needCount`("지금 넣어야 할 상품 6개")를 보여주고 S03으로, FAILED면 M3 실패 팝업에 `errorMessage`를 보여준다. 실패해도 이전 결과(`previousRunId`)는 그대로 조회되고 D1에서 "갱신 전"으로 표시된다.

3.5 입고 필요 목록 (D1)

`GET /dashboard/replenishment-needs?sort=shortage | risk | amount` (소진일 빠른 순 · 품절 위험 높은 순 · 금액 큰 순, 기본 shortage). 열 때마다 서버가 오늘 날짜로 입고 필요 판정을 다시 맞춘다.

```
{"asOfDate": "2026-10-14", "today": "2026-10-19", "daysSinceAsOf": 5,
 "resultStatus": "UP_TO_DATE", "runId": 34, "resultComputedAt": "2026-10-
19T09:00:21+09:00", "lastFailedRunAt": null, "latestUploadId": 15,
 "summary": {"needCount": 6, "notNeededCount": 22, "notCalculableCount": 2,
"excludedCount": 1},
 "items": [{"productId": 101, "resultId": 9001, "skuBarcode":
"8800272420016", "skuName": "케어트릿 120g",
   "recommendedQty": 72, "recommendedAmount": 129600, "expectedShortageOn":
"2026-10-22", "leadTimeDays": 5, "preArrivalRisk": 0.04}],
 "notCalculable": [{"productId": 121, "skuName": "시즌 간식 세트", "reason": "
판매한 날(유효 관측일)이 7일보다 적어요"}],
 "excluded": [{"productId": 108, "skuName": "덴탈껌 M 5입", "date": "2026-09-
12", "reason": "재고 숫자가 맞지 않아요"}]}
```

판정 기준: 예상 소진일이 오늘 + 리드타임 안이면 입고 필요(기준일~오늘은 예측 출고로 이어서 계산). `resultStatus`는 `UP_TO_DATE` · `STALE`이며, 마지막 계산이 실패했으면 `STALE`이고 화면은 "갱신 전"을 표시한다. `notCalculable` · `excluded`는 대시보드 아래 목록, `latestUploadId`는 "검증 리포트 보기" 링크(U2)에 쓴다. `daysSinceAsOf`가 7을 넘으면(기획서 N = 7일) 계산하지 않고 재업로드를 안내하며, 이때 SKU 결과 status는 REUPLOAD_REQUIRED다. STALE이면 lastFailedRunAt(실패한 계산 시각)과 resultComputedAt(지금 보여주는 결과의 계산 시각)으로 "마지막 계산(10월 19일 오전 9시)이 실패해서 10월 18일 계산 결과를 보여주고 있어요"를 표시한다.

3.6 SKU 상세 (K1)

`GET /products/{productId}/detail?from=2026-07-01&to=2026-10-14`


```
{"product": {"productId": 101, "skuBarcode": "8800272420016", "skuName": "
케어트릿 120g", "fulfillment": "풀필먼트 A사"},
 "needStatus": "NEED_NOW", "asOfDate": "2026-10-14", "expectedShortageOn":
"2026-10-22",
 "settings": {"leadTimeDays": 5, "leadTimeSource": "DEFAULT",
"reviewCycleDays": 29, "reviewCycleSource": "ESTIMATED", "orderUnitQty": 12,
"unitCost": 1800},
 "result": {"resultId": 9001, "status": "OK", "method": "A", "methodReason":
"평가 결과 기본 방식", "multiplier": 1.05, "horizonDays": 34,
   "forecastOutboundQty": 102, "startStockQty": 40, "startStockEstimated":
true, "plannedReceiptsInHorizon": 0, "preArrivalUnmetQty": 0,
   "requiredQtyBeforeRounding": 67, "recommendedQty": 72,
"recommendedAmount": 129600},
 "accuracy": {"grade": "HIGH", "isReference": false, "validObservationDays":
412, "coverSuccessCount": 9, "coverTrialCount": 10, "forecastErrorRate":
0.12, "history": [{"originDate": "2026-08-01", "forecastQty": 98,
"actualQty": 101, "covered": true}], "range": null, "firstInboundOn": "2024-
11-02"},
 "trend": [{"date": "2026-09-08", "closingQty": 0, "outboundQty": 0,
"isValidObservation": false},
           {"date": "2026-10-14", "closingQty": 55, "outboundQty": 3,
"isValidObservation": true}]}
```


`needStatus` 는 `NEED_NOW`(지금 넣어야 해요) · `NOT_NEEDED`(아직 괜찮아요) · `NOT_CALCULABLE`(계산 불가, `reason` 필수) · `EXCLUDED`(검산 제외) · REUPLOAD_REQUIRED(재업로드 필요, 기준일이 7 일 넘게 지남). `settings` 는 화면 아래 입고 조건 카드, `requiredQtyBeforeRounding` 은 "필요한 양 67 개 → 입고 단위로 올려 72 개" 줄에 쓴다. `trend` 에서 `isValidObservation` 이 false 인 날(시작 재고 0 일)은 차트에 빨간 점으로 표시하고 계산에서 뺀다. accuracy 는 "이 숫자, 얼마나 믿을 수 있나요" 카드에 쓴다(FE-09 · DA-13). grade 는 HIGH(높음) · MEDIUM(보통) · LOW(낮음)이고, coverSuccessCount/coverTrialCount 는 "10 번 중 9 번 버텼어요", history 는 과거 예측 대 실제 출고 그래프(covered 가 false 면 "빗나감")다. LOW 면 isReference: true 로 "참고용"을 표시하고 버틴 횟수 대신 range(예측이 흔들릴 수 있는 범위, low · high)를 준다. 유효 관측일이 7 일 미만이면 grade 없이 needStatus 가 NOT_CALCULABLE 이다. 등급 기준은 DA-13(팀 결정 예시)을 따른다. startStockQty 는 기준일 종료 재고(예: 10/14 55 개)에서 오늘까지의 예상 출고를 빼고 기존 발주 도착분을 더한 시작 재고(예: 40 개)이고, startStockEstimated 가 true 면 오늘이 기준일보다 늦어 추정한 값이므로 화면 근거 카드에 "시작 재고(추정)"로 표시한다(기획서 시작 재고 규칙, FE-05).

3.7 시나리오 (K2 · K3)


```
// K2 GET /results/9001/scenarios
{"resultId": 9001, "isEstimate": true,
 "preArrivalRisk": 0.04, "riskSampleSize": 38, "riskStatus": "OK",
 "scenarios": [
  {"scenarioId": 1, "type": "P80",  "qty": 60, "isDuplicate": false,
"expectedDepletionOn": "2026-11-18",
   "expectedEndingStock": 0,  "unmetDemandQty": 6, "postArrivalRisk": 0.25},
  {"scenarioId": 2, "type": "P100", "qty": 72, "isDuplicate": false,
"expectedDepletionOn": null,
   "expectedEndingStock": 6,  "unmetDemandQty": 0, "postArrivalRisk": 0.08},
```

```
  {"scenarioId": 3, "type": "P120", "qty": 96, "isDuplicate": false,
"expectedDepletionOn": null,
   "expectedEndingStock": 30, "unmetDemandQty": 0, "postArrivalRisk": 0.02}]}

// K3 POST /results/9001/scenarios
{"qty": 84}
// → 201 {"scenarioId": 4, "type": "CUSTOM", "qty": 84, ...}
```

- 표본이 최소 기준에 못 미치면 위험값은 `null`, `riskStatus: "INSUFFICIENT_SAMPLE"` → 화면은 "산출 불가 + 표본 수"를 보여준다. 정확도 등급이 LOW 면 위험값은 null, riskStatus: "LOW_GRADE"이고 화면은 "기록이 짧아 산출하지 않았어요"를 보여준다(DA-07).
- 입고 단위로 올린 뒤 수량이 같으면 `isDuplicate: true`. 권장량이 0 개면 `scenarios` 는 비어 있고 K3(직접 입력)만 쓴다.
- `qty` 가 1 미만이면 `400 INVALID_INPUT`. 직접 입력은 결과 1 건에 1 개만 저장하고, 새로 입력하면 이전 직접 입력 시나리오를 덮어쓴다.

3.8 AI 인사이트 (K4)

`GET /results/{resultId}/insight`

```
// 공개 응답 형식 예시(실측 결과 아님)
{"resultId":9001,"mode":"AI","isFallback":false,"text":"계산값으로 검증된 설명
문자열","failReason":null,"generatedAt":"2026-01-01T09:00:00+09:00"}
```

- `mode`: `AI`(AI 작성 문장) · `FIXED`(고정 문장) · `TABLE_ONLY`(비교표만). 실패로 고정 문장이 쓰이면 `isFallback: true` → 화면에 "기본 문장"을 작게 표시(10/19 결정).
- 계산불가·입고불필요·검산제외·재업로드필요는 FastAPI 를 부르지 않고 Spring 의고정문장으로 응답하며,필수사유/값이없으면 TABLE_ONLY/text=null 일수있다.
- 준비/처리중은 202,완료는 200 이며 text 는 null 일수있다. 2026-11-05 사용자 결정으로 Server PR124 에서 nullable failReason 을 추가한다(기존 b66bc8a 배포에는 없음). 실패 대체이면 LLM_ERROR/LLM_TIMEOUT/VALIDATION_FAILED/VALUE_MISMATCH/LLM_NOT_CONFIGURED/INVALID_INPUT/INPUT_UNAVAILABLE 중 하나이며, 알 수 없는 내부 사유는 LLM_ERROR 로 축약한다. 정상·비활성·상태별 고정 문장은 null 이다. reasonCode/values·원문 오류는 공개하지 않으며 DB 원래 사유는 보존한다. isFallback 만으로 실제유료호출 여부를추정하지않는다. AI 실패는계산결과를훼손하지않는다.

3.9 정확도 (K1 에 포함)

별도 정확도 평가 화면(S08)과 E1 API 는 두지 않는다(FE-09). SKU 별 정확도는 K1 의 accuracy 로 내려준다. 정책 평가(A/B 오차 · 배수별 지표 · 실제 입고 비교)는 팀 내부에서 /engine/backtest 로 실행하고 결과는 내부 검증용으로 저장만 한다(BE-12 · DA-08~DA-12).

4. 백엔드(Spring Boot) ↔ FastAPI

현재 정제·계산은 Spring outbox/SQS 비동기 경로이며 직접 수량·AI 설명은 내부 HTTP 경로다. FastAPI 는 승인된 DB view 를 조회하고 결과 저장·공개 API·권한·값 치환은 Spring 이 담당한다. 아래는 2026-11-05 Server b66bc8a·AI 27a0063 구현과 대조한 반영안이며 최종 공동 확인은 Server #25 / AI #24·#44 에서 남긴다.

호출	Spring → FastAPI (요청)	FastAPI → Spring (응답)	현행 시간·재시도 정책
SQS CLEAN 정제·검증	contractVersion=1, jobId, jobType=CLEAN, payload: uploadId/userId/warehouseId/previousUploadFileId 및 files 1개 (uploadFileId/storageKey 등)	기간·기준일, 상품(새 바코드), 재고관리단위, 재고관리단위별 일별 수불, 상품 일별 수불(유효 관측일 포함), 검증 결과(제외 SKU·사유)	서버 재전송 20분·만료 2시간; lease60초
SQS CALCULATE 입고 계산	contractVersion=1, jobId, jobType=CALCULATE, payload: runId/userId/warehouseId/asOfDate/today/productIds/uploadFileIds/policy/settings/plannedReceipts/noDateReceipts	SKU 별 결과 + 80·100·120% 시나리오	서버 재전송 5분·만료 15 분; lease60초
POST /engine/scenarios 직접 입력	resultId 의 계산 입력, qty	시나리오 1 개	10 초
POST /engine/backtest 정확도 평가 (서버 연결은 별도 미완료)	userId, 조정·시험 기간, 배수 후보 [1.00, 1.05, 1.10, 1.15]	기본 방식·배수와 사유, A·B 오차, 배수별 지표, 실제 입고 대비, 결과별 백테스트 지표, SKU 별 정확도 등급·버틴 횟수·예측 대 실제 이력(DA-13)	300 초
POST /engine/insight AI 인사이트	resultId, 구조화된 시나리오 결과(권장 시나리오 · 자리 표시자 목록 · 비교 방향 · 위험 상태)	검증 결과(PASSED / FAILED), 검증된 문장 초안, 실패 사유, 응답시간, 호출 비용	15 초

```
// SQS CALCULATE payload 예시{"runId": 34, "userId": 1, "asOfDate": "2026-10-14", "today": "2026-10-19", "productIds": [101], "policy": {"defaultMethod": "A", "multiplier": 1.05}, "settings": [{"productId": 101, "leadTimeDays": 5, "reviewCycleDays": 29, "orderUnitQty": 12, "unitCost": 1800}], "plannedReceipts": [{"productId": 101, "quantity": 120, "expectedOn": "2026-12-01"}]}// 응답{"engineVersion": "0.3.1", "results": [{"productId": 101, "status": "OK", "reason": null,   "method": "A", "methodReason": "평가 결과 기본 방식", "multiplier": 1.05, "horizonDays": 34,   "forecastOutboundQty": 102, "startStockQty": 40, "startStockEstimated": true, "plannedReceiptsInHorizon": 0, "preArrivalUnmetQty": 0, "requiredQtyBeforeRounding": 67, "recommendedQty": 72, "recommendedAmount": 129600, "expectedShortageOn": "2026-10-22",   "preArrivalRisk": 0.04, "riskSampleSize": 38, "riskStatus":
```

```
"OK", "accuracyGrade": "HIGH", "validObservationDays": 412,
"coverSuccessCount": 9, "coverTrialCount": 10, "forecastErrorRate": 0.12,
"scenarios": [{"type": "P100", "qty": 72, "isDuplicate": false,
"expectedDepletionOn": null,     "expectedEndingStock": 6, "unmetDemandQty":
0, "postArrivalRisk": 0.08,     "dailyProjection": [{"date": "2026-10-15",
"closingQty": 37}]}]}]}
```


규칙

- 정제·계산 결과는 warehouse→job→upload/run 잠금과 transaction 으로 저장하고 commit 뒤 결과 ACK 한다. 전송/저장 재시도는 같은 jobId 를 사용하며 확정 FAILED/DONE 뒤 늦은 결과는 무시한다. 실패/시간초과·재시도·실제 화면 검증 근거는 Server #95 에서 구분한다.
- 계산 불가는 실패가 아니다. `status: "NOT_CALCULABLE"`과 `reason`으로 돌려주고, 0 개로 바꾸지 않는다.
- 계산 요청과 파일 출처·버전을 보존한다. run.selection_snapshot 에 원래 요청을, result.input_snapshot 에 상품별 요청을 저장한다. 설명이 준비된 경우 ai_explanations.request_payload 에 input/values/fallback 을 보관하며 비활성 행의 payload 는 비어 있을 수 있다.
- `today`는 Spring 이 넣는다. FastAPI 는 기준일 다음 날부터 오늘까지를 예측 출고로 이어서 입고 필요를 판정한다. 오늘이 기준일보다 7 일 넘게 지났으면 Spring 은 계산 엔진을 부르지 않고 재업로드를 안내한다(기획서 N = 7 일).
- `/engine/insight 실패·시간초과·검사 탈락은 Spring 이 계산값 기반 기본 문장 또는 표시 생략으로 마감하며 자동 유료 재시도하지 않는다. 과거 DISABLED 소급 생성·CUSTOM 변경 후 설명 재생성·모델/실제 token 기록 확대는 현재 범위 밖이다.`

4.1 설명·시나리오의 현재 경계 (공동 확인 대기)

FastAPI 는 validation/draft/failReason/promptVersion/latencyMs/costUsd 를 반환한다. mode/templateVersion/failDetail 은 내부 응답 필드가 아니다. Spring 은 reason_code·versions(template=spring-1, prompt=응답버전)·latency_ms/cost_usd 를 저장한다. 모델/실제 token 확대는 Server #33 에서 추적한다.

자리 표시자 이름과 비교 방향은 FastAPI 초안 검사의 대상이며 실제 값 대입·표시 정밀도·수치 대조와 기본 문장은 Spring 책임이다. 미산출 위험을 0 이나 임의 표본부족으로 채우지 않는다.

공개 K4 는 resultId/mode/isFallback/text/generatedAt 이며 text 는 단일문자열 또는 null 이다. mode=AI/FIXED/TABLE_ONLY 가 DB 의 LLM/TEMPLATE/NONE 과 대응한다. 준비/처리중은 202, 완료는 200 이다.

NO_DATE 발주는 noDateReceipts 로 전달해 설명의 사실만 표시하고 예정 입고량으로 합산하지 않는다.

직접 수량은 POST /api/results/{resultId}/scenarios 에 {qty},201·단일 CUSTOM 이다. resultId 에서 소유자/상품/원래 run/source 를 확인하고 입고단위로 올린다. 같은수량은동일 ID, 다른동시수량은 409 이며 isDuplicate 는 기본비율과 수량중복 여부다.

CUSTOM 은 기본 추천·기본비교표·원래입력·기존설명을 보존한다. null/부적절수량 400·오래된결과 409/SCENARIO_STALE,엔진오류 502·미사용/저장실패 503·시간초과 504 를 구분한다. CUSTOM 설명 재생성 시점/범위는 미정 후속이며 이번 문서가 새 기능을 승인하지 않는다.

코드·실측·기존합의 대조: IpgoITGO/ipgo-server docs/decisions/0009-current-insight-scenario-contract.md 및 Server25, AI24/44, Client78. 이 명세 반영안은 담당자의 최종 수락이나 실제 서비스 추가검증을 대신하지 않는다.

5. AI 인사이트: FastAPI ↔ LLM

AI 인사이트는 왜 이 수량을 추천하는지를, 줄이거나 늘렸을 때 달라지는 점과 함께 한두 문장으로 짚어준다. LLM 은 숫자 자리를 자리 표시자로 비워 둔 **문장 초안**만 쓰고, 검증을 통과한 초안에 Spring 이 계산값을 넣은 것이 AI 인사이트다.

단계	어디서	하는 일
1	Spring	계산 commit 이후 승인된 설명 작업에서 /engine/insight 를 호출하고 검증·치환·저장한다. K4 GET 은 저장된 설명을 조회한다. 계산불가·불필요·검산제외·재업로드필요는 FastAPI 호출 없이 Spring 의 고정 문장 또는 비교표만 응답한다.
2	FastAPI	프롬프트 구성 (데이터/AI 가 정한 규칙, 해당 SKU 결과만)
3	LLM	문장 초안 작성 (숫자 자리는 자리 표시자)
4	FastAPI	자리 표시자 검증 (자리 밖 숫자 · 평가 어휘 · 필수 자리 · 형식)
5	Spring	검증된 초안의 자리 표시자에 계산값 넣기 (위험은 %, 수량은 "개")
6	Spring	넣은 값과 계산 결과 대조, `ai_explanations` 에 저장
7	화면	AI 인사이트 표시. 기본 문장이면 작게 표시

```
// FastAPI → LLM (프롬프트에 넣는 데이터)
{"skuLabel": "상품 A", "recommended": "P100",
 "placeholders": ["qty_p80", "qty_p100", "qty_p120", "ending_p100",
"risk_post_p80", "risk_post_p100", "risk_pre"],
 "directions": {"risk_post_p80_vs_p100": "높음", "ending_p120_vs_p100": "높음"},
 "riskStatus": "OK"}

// LLM → FastAPI (문장 초안)
{"text": "권장량 {qty_p100}개는 다음 입고 때 {ending_p100}개만 남기면서 품절 위험을
{risk_post_p100}로 낮춰요. {qty_p80}개로 줄이면 품절 위험이 {risk_post_p80}로
```

```
올라가요."}

// FastAPI → Spring (/engine/insight 응답)
{"validation": "PASSED", "draft": "권장량 {qty_p100}개는 ...", "failReason":
null, "latencyMs": 1840, "costUsd": 0.0021}
```

- LLM 에 보내지 않는 것: 원본 수불부, 원가, 계정 정보, 다른 SKU 결과. 실제 숫자를 함께 보낼지는 에이전트 구성도 11 장 1 번에서 정한다(위 예시는 보내지 않는 쪽).
- 필수 자리 표시자(제안): `qty_p100`, `risk_post_p100`, `risk_pre`. 빠지면 검증 실패.
- 입고 불필요(0 개) SKU 도 계산 불가와 같이 LLM 을 부르지 않고 고정 문장을 쓴다(BE-10).

실패	어디서 걸리나	처리	`fail_reason`
LLM 호출 오류	FastAPI	기본 문장	`LLM_ERROR`
시간 초과 (LLM 10 초 [가정])	FastAPI	기본 문장	`LLM_TIMEOUT`
자리 표시자 검증 실패	FastAPI	기본 문장	`VALIDATION_FAILED`
넣은 값 불일치	Spring	기본 문장	`VALUE_MISMATCH`

어느 단계든 실패하면 같은 숫자로 기본 문장을 보여주고 다시 부르지 않는다(단일 호출). Spring 은 호출마다 `ai_explanations` 에 문장 초안(`raw_text`), 최종 문장(`rendered_text`), 검증 통과 여부, 실패 사유, 응답시간(`latency_ms`), 호출 비용(`cost_usd`)을 저장한다(BE-13 · BE-16).

6. 상태 코드 · 오류 코드

오류는 HTTP 상태와 code 로 구분하고, 화면은 code 에 따라 안내 문구를 고른다(FE-13). 계산 불가와 AI 대체는 오류가 아니라 정상 응답 안의 상태값이다. 아래 인증 오류 표에서 갱신 가능 여부를 구분한다.

HTTP	`code`	언제	화면 처리
400	`INVALID_INPUT`	필수값 누락, 0 이하 수량, 정수가 아닌 입고 단위	해당 칸 옆에 안내 (`details.field`)
400	`UPLOAD_INVALID_FORMAT`	XLSX 아님, 파일명에서 연·월을 못 읽음	파일별 거부 사유
401	`UNAUTHENTICATED`	보호 API 요청에 인증 토큰이 없음	로그인 화면으로 이동
401	`INVALID_CREDENTIALS`	이메일·비밀번호 불일치	로그인 화면 안내
401	ACCESS_TOKEN_EXPIRED	액세스 JWT 만료	refresh 1 회 후 원래 요청 1 회 재시도. 실패하면 재로그인
401	INVALID_ACCESS_TOKEN	잘못되거나 변조된 JWT	토큰 제거 후 재로그인

HTTP	code	언제	화면 처리
401	SESSION_REVOKED	로그아웃·폐기된 로그인	토큰 제거 후 재로그인
401	INVALID_REFRESH_TOKEN	누락·알 수 없거나 폐기된 refresh	재로그인
401	REFRESH_TOKEN_EXPIRED	최초 로그인부터 7 일 마감 도달	재로그인
401	REFRESH_TOKEN_REUSED	이전 refresh 재사용으로 해당 로그인 폐기	재로그인
403	FORBIDDEN	다른 계정의 데이터 요청 또는 쿠키 기반 인증 POST 의 CSRF 누락·오류	접근 불가 안내
404	NOT_FOUND	없는 상품·업로드·결과	빈 결과 안내
409	EMAIL_EXISTS	이미 가입한 이메일	가입 화면 안내
400	VERIFICATION_FAILED	인증 코드가 틀림 (5 번 틀리면 코드 무효)	칸 옆에 안내, 코드 다시 받기
400	VERIFICATION_EXPIRED	인증 코드 10 분 또는 인증 토큰 30 분이 지남	코드 다시 받기 안내
400	VERIFICATION_REQUIRED	인증 토큰 없이 가입 요청	이메일 인증 먼저 안내
429	TOO_MANY_REQUESTS	인증 메일을 1 분 안에 다시 요청	잠시 후 다시 보내기 안내
409	UPLOAD_DUPLICATE	같은 기간을 다시 올림	덮어쓰기 확인 팝업
409	RUN_IN_PROGRESS	같은 계산이 이미 실행 중	기존 작업 상태를 보여줌
502	ENGINE_ERROR	계산 엔진 실패	다시 시도 안내, 이전 결과 유지
504	ENGINE_TIMEOUT	계산 엔진 시간 초과	다시 시도 안내, 이전 결과 유지

정상 응답 안의 상태값

대상	값
업로드 `status`	`PROCESSING` · `DONE` · `FAILED`
계산 `status`	`QUEUED` · `RUNNING` · `DONE` · `FAILED`
SKU 결과 `status`	`OK` · `NOT_NEEDED` · `NOT_CALCULABLE` · `EXCLUDED` · `REUPLOAD_REQUIRED`

대상	값
품절 위험 `riskStatus`	`OK` · `INSUFFICIENT_SAMPLE` · `LOW_GRADE`
AI 인사이트 `mode`	`AI` · `FIXED` · `TABLE_ONLY` (+ `isFallback`)
정확도 grade	`HIGH · MEDIUM · LOW` (유효 관측일 7일 미만은 등급 없이 계산 불가)
기존 발주 status	`PENDING · RECEIVED · OVERDUE · NO_DATE · CANCELLED`

7. 팀 확정 필요 사항

아래가 정해지면 해당 절의 "예시·제안" 표시를 지운다. Python 쪽 통신 방식은 FastAPI · HTTP · JSON 으로 정해졌다.

- ☑ 인증 방식 확정: 보호 API 는 JWT Bearer, 기본 액세스 15 분·최초 로그인부터 리프레시 최대 7 일. HttpOnly 리프레시 쿠키·회전·로그아웃 폐기·쿠키 POST 의 CSRF·만료 화면 처리 계약은 1 장·3.1·6 장에 반영 — 백엔드·프론트
- ☐ 연동 명세 검토: 4 장 `calculate` · `insight` 응답 필드(시나리오 결과 형식 포함)를 기준안으로 확정 — 백엔드 + 데이터/AI
- ☐ AI 인사이트 세부: 실제 숫자 전송 여부, 필수 자리 표시자, 시간 제한, 생성 시점(화면을 열 때마다 / 계산 ID 당 1 회 저장) (5 장, 3.8) — 데이터/AI
- ☑ 입고 불필요(0 개) SKU 의 LLM 생략: 생략으로 확정 (요구사항 BE-10 최종, 3.8 · 5 장)
- ☑ 직접 입력 수량: 새 계산 ID 를 만들지 않고 같은 결과 아래 시나리오로 붙임. 결과 1 건에 1 개, 새로 입력하면 덮어씀 (ERD 중복 금지 조건, 3.7)
- ☑ DB: PostgreSQL (최종 ERD AX_project_v6, 표 16 개)
- ☐ 업로드 묶음: 파일 여러 개를 uploadId 하나로 묶는 방식. ERD 의 upload_files 는 파일마다 1 행이므로 job_id 를 묶음 번호로 쓸지 정하기 (3.2) — 백엔드